

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Debugging & Optimisation Task

Course Code:	CSA0620	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO2 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	Kamalesh T	Reg. No.:	192424228
Section / Batch:	2024	Date:	31/07/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1	100
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. Debugging Selection Sort Code (Minimum Index Compared Against Wrong Loop Variable in C)

The following Selection Sort implementation in C compares the candidate element against `arr[j]`

instead of `arr[min_idx]`, which prevents the algorithm from preserving the true minimum seen so

far within the current pass. Carefully analyze why the minimum candidate must be updated relative

to the current best position, not the current scanning position. Apply systematic debugging so that

the smallest element of the unsorted region is tracked correctly. Optimize the corrected implementation by skipping unnecessary swaps when no movement is needed. Analyze the time

complexity of the corrected version and rewrite the final optimized C code with proper comments.

```
void selectionSort(int arr[], int n) {  
    int i, j, min_idx, temp;  
    for (i = 0; i < n - 1; i++) {  
        min_idx = i;
```

```

for (j = i + 1; j < n; j++) {
    if (arr[j] < arr[min_idx]) { // ERROR
        min_idx = j;
    }
}
temp = arr[i];
arr[i] = arr[min_idx];
arr[min_idx] = temp;
}
}

```

Code of Conduct

I P. Kamalesh T(192424228) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Kamalesh T_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (100 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Debugging Selection Sort Code (C)

Aim

To identify and fix the bug in the given Selection Sort program, optimize it by avoiding unnecessary swaps, analyze its time complexity, and provide the corrected C program.

Problem Analysis

The given Selection Sort program contains the following incorrect statement:

```
if (arr[j] < arr[j])    // ERROR
```

This condition compares the same element with itself.

Since `arr[j]` is always equal to `arr[j]`, the condition is **always false**, so `min_idx` is never updated.

As a result, the algorithm fails to identify the smallest element in the unsorted portion of the array.

Why Compare with `arr[min_idx]`?

Selection Sort must always remember the **smallest element found so far**.

- `min_idx` stores the index of the current minimum element.
- During scanning, every new element should be compared with the current minimum (`arr[min_idx]`).
- If a smaller element is found, `min_idx` is updated.

Correct comparison

```
if (arr[j] < arr[min_idx])
```

This ensures that the minimum element found so far is preserved throughout the pass.

Systematic Debugging Steps

Step 1

Locate the incorrect comparison.

Incorrect

```
if (arr[j] < arr[j])
```

Step 2

Replace it with

```
if (arr[j] < arr[min_idx])
```

Now the smallest element is tracked correctly.

Step 3

Optimize the algorithm.

If the minimum element is already at position `i`, swapping is unnecessary.

Add:

```
if (min_idx != i)
```

before performing the swap.

This avoids unnecessary data movement.

Corrected and Optimized C Program

```
#include <stdio.h>
// Function to perform Selection Sort
void selectionSort(int arr[], int n)
{
    int i, j, min_idx, temp;

    // Traverse the array
    for(i = 0; i < n - 1; i++)
    {
        // Assume first element is minimum
        min_idx = i;

        // Find the smallest element
        for(j = i + 1; j < n; j++)
        {
            if(arr[j] < arr[min_idx])
            {
                min_idx = j;
            }
        }

        // Swap the minimum element with the element at index i
        temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}
```

```

        // Swap only if needed
        if(min_idx != i)
        {
            temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx] = temp;
        }
    }
}
// Function to print array
void printArray(int arr[], int n)
{
    int i;

    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
}
int main()
{
    int arr[] = {28, 16, 35, 11, 19};

    int n = sizeof(arr)/sizeof(arr[0]);

    printf("Original Array:\n");

    printArray(arr, n);

    selectionSort(arr, n);

    printf("\n\nSorted Array:\n");

    printArray(arr, n);

    return 0;
}

```

Sample Output

Original Array:
28 16 35 11 19

Sorted Array:
11 16 19 28 35

Time Complexity Analysis

Case	Time Complexity	Explanation
Best Case	$O(n^2)$	All comparisons are still performed, even if the array is

Case	Time Complexity	Explanation
		already sorted.
Average Case	$O(n^2)$	Nested loops examine all unsorted elements.
Worst Case	$O(n^2)$	Every pass scans the remaining unsorted elements.
Space Complexity		
$O(1)$ (In-place sorting algorithm)		

Optimization Analysis

The optimization:

```
if(min_idx != i)
```

does **not** reduce the number of comparisons, but it **reduces unnecessary swaps**.

Without Optimization

- Maximum swaps = $n - 1$

With Optimization

- Swaps occur **only when the minimum element is not already in its correct position**, reducing unnecessary data movement and improving practical performance.

Conclusion

The original program failed because it compared `arr[j]` with itself instead of the current minimum element `arr[min_idx]`, preventing the algorithm from updating the minimum index correctly. Replacing the comparison with `arr[j] < arr[min_idx]` fixes the bug. Adding a condition to skip swaps when `min_idx == i` further optimizes the implementation by avoiding unnecessary data movement. The corrected Selection Sort remains an in-place algorithm with $O(n^2)$ time complexity and $O(1)$ space complexity while producing the correct sorted output.